

Chapter 3: Information Embedding

Complete Repository & Textbook Code Mapping Guide | RAG from First Principles

1. CHAPTER 3 OVERVIEW & COMPLETE FILE INVENTORY

Chapter 3 of *RAG from First Principles* explores **Information Embedding**—the core mechanism for encoding text, structured metadata, and multimodal inputs into high-dimensional vector spaces. The repository folder `03-Embedding/` contains **8 total items** (6 standalone Python scripts, 1 environment configuration template, and 1 PyPI requirements manifest).

File Name	Category / Type	Primary Purpose & Technique	Textbook Mapping
<code>.env.example</code>	Config Template	Environment variables template for OPENAI_API_KEY and JINA_API_KEY.	Config Setup (p. 113)
<code>requirements.txt</code>	Dependency List	Manifest for torch, FlagEmbedding, rank-bm25, chromadb, scikit-learn, etc.	Setup (p. 113)
<code>01-openai-embedding-recomendation-system.py</code>	Standalone Script	Product recommendation system using OpenAI text-embedding-3-small & Cosine Similarity.	pp. 113–115
<code>02-jina-embeddings-v3-clustering.py</code>	Standalone Script	Cross-lingual game description clustering using Jina v3 API & sklearn KMeans.	pp. 115–118
<code>03-BM25.py</code>	Standalone Script	From-scratch mathematical implementation of BM25 sparse keyword embeddings.	pp. 123–126
<code>03-LangChain-BM25.py</code>	Standalone Script	Turnkey sparse keyword retrieval using LangChain's BM25Retriever component.	p. 126
<code>04-BGE-M3.py</code>	Standalone Script	Multi-functional hybrid embeddings with BGE-M3 (Dense, Sparse, ColBERT multi-vector).	pp. 126–128
<code>05-MultimodalEmbedding.py</code>	Standalone Script	Cross-modal text + image embedding using FlagEmbedding's Visualized_BGE model.	pp. 128–129

2. DETAILED PURPOSE & MECHANICS OF REPOSITORY SCRIPTS

• **01-openai-embedding-recomendation-system.py (pp. 113–115)**

Demonstrates commercial dense vector generation using OpenAI's text-embedding-3-small (1536 dims). It generates embedding vectors for user review texts and game catalog descriptions, then computes cosine similarity scores to recommend the top matching games for each user profile.

• **02-jina-embeddings-v3-clustering.py (pp. 115–118)**

Uses Jina AI's jina-embeddings-v3 REST API to extract 1024-dimensional vectors for multilingual game descriptions (Chinese, English, German). It applies scikit-learn's KMeans algorithm to group game descriptions semantically into 3 clusters regardless of their input language.

• **03-BM25.py (pp. 123–126)**

A pure Python implementation of the Okapi BM25 sparse keyword scoring formula. It tokenizes raw combat logs, computes Inverse Document Frequency (IDF), applies term frequency (TF) scaling with hyperparameters $k1=1.5$ and $b=0.75$, and outputs a sparse dictionary mapping vocabulary indices to BM25 weights.

• **03-LangChain-BM25.py (p. 126)**

Demonstrates high-level sparse keyword retrieval using LangChain's BM25Retriever.from_documents(). It abstracts away manual IDF and TF computations, allowing instant integration into LangChain RAG pipelines.

• 04-BGE-M3.py (pp. 126–128)

Demonstrates BAAI's tri-mode bge-m3 model via FlagEmbedding.BGEM3FlagModel. A single forward pass returns **Dense vectors** (1024 dims), **Sparse lexical weights** (term-importance dictionary), and **ColBERT Multi-vectors** (token-level representations) simultaneously for hybrid search.

• 05-MultimodalEmbedding.py (pp. 128–129)

Implements joint image-text cross-modal encoding using Visualized_BGE (base EN v1.5). It projects visual features (PIL Images) and text queries into a shared vector space, enabling visual search and multimodal RAG.

3. TEXTBOOK CODE SNIPPETS (NOT STANDALONE FILES IN REPO)

In addition to the standalone .py files in 03-Embedding/, Chapter 3 of the textbook presents **4 crucial code listings** directly in the text body to explain vector math, framework abstractions, and embedding caching:

A. Cosine Similarity Vector Calculation (Textbook p. 101)

Purpose: Mathematical baseline function using NumPy to compute the cosine of the angle between two embedding vectors.

```
import numpy as np

def cosine_similarity(vec1, vec2):
    dot_product = np.dot(vec1, vec2)
    norm_vec1 = np.linalg.norm(vec1)
    norm_vec2 = np.linalg.norm(vec2)
    return dot_product / (norm_vec1 * norm_vec2)

# Usage example:
similarity = cosine_similarity(query_vector, doc_vector)
print("Cosine Similarity:", similarity)
```

B. LangChain OpenAIEmbeddings Interface (Textbook pp. 130–131)

Purpose: Universal LangChain wrapper interface for calling commercial embedding APIs with query/document batching.

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small" # Options: text-embedding-3-large, text-embedding-ada-002
)

# Embed a single query string:
query_vector = embeddings.embed_query("What skills does Sun Wukong possess?")
print(f"Vector dimension: {len(query_vector)}")

# Embed a batch of document strings:
doc_vectors = embeddings.embed_documents(["Wukong uses Flaming Fist.", "Bajie uses rake."])
```

C. LlamaIndex Global Embedding Settings (Textbook p. 131)

Purpose: Configuring a global HuggingFace/local embedding model across LlamaIndex indexes via Settings.

```

from llama_index.core import SimpleDirectoryReader, VectorStoreIndex, Settings
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

# Configure global embedding model:
Settings.embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-zh")

# Load documents & build vector index automatically using global embed_model:
documents = SimpleDirectoryReader("data/game_lore/").load_data()
index = VectorStoreIndex.from_documents(documents)

query_engine = index.as_query_engine()
response = query_engine.query("Summarize the game lore.")

```

D. LangChain Embedding Caching with CacheBackedEmbeddings (Textbook pp. 131–133)

Purpose: Persistent disk/key-value caching of embedding vectors to avoid duplicate API costs and execution latency.

```

from langchain.embeddings import CacheBackedEmbeddings
from langchain.storage import LocalFileStore
from langchain_openai import OpenAIEmbeddings

# Initialize local key-value file store for cached vectors:
store = LocalFileStore("./cache_embeddings/")

underlying_embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# Wrap underlying model with persistent cache:
cached_embedder = CacheBackedEmbeddings.from_bytes_store(
    underlying_embeddings,
    store,
    namespace=underlying_embeddings.model
)

# First run computes vectors and caches to disk; second run fetches instantly from store:
vector = cached_embedder.embed_query("Query string to embed")

```

4. DEVELOPER DECISION MATRIX: EMBEDDING STRATEGY SELECTION

Selecting the right embedding strategy is critical for balancing retrieval recall, precision, computational cost, and hardware constraints:

Embedding Strategy	Primary Strengths	Trade-offs / Limitations	Matched File / Code
Dense OpenAI (text-3-small)	High semantic recall, easy API integration, multi-lingual support.	Requires internet/API keys; recurring token cost (\$0.02/1M tokens).	01-openai-...py & LangChain wrapper
Jina Embeddings v3	XLNet-RoBERTa based, supports 8192 token long-contexts & Matryoshka dims.	API dependency; requires task parameter tuning ('text-matching', 'retrieval').	02-jina-embeddings-v3-clustering.py
Sparse Keyword (BM25)	Exact term matching (part numbers, technical jargon, proper names).	No semantic understanding (fails on synonyms or paraphrasing).	03-BM25.py & 03-LangChain-BM25.py
Hybrid BGE-M3 (Tri-Mode)	Simultaneous Dense + Sparse + ColBERT multi-vector representation.	Higher memory footprint; requires FlagEmbedding/PyTorch setup.	04-BGE-M3.py
Visualized-BGE (Multimodal)	Joint image and text vector encoding into a single unified space.	Requires heavy PyTorch weights (Visualized_base_en_v1.5.pth) and GPU.	05-MultimodalEmbedding.py
Cache-Backed Embeddings	Eliminates redundant embedding API calls and reduces pipeline latency.	Requires local disk/KV storage management and invalidation rules.	Textbook pp. 131–133 code snippet